

- ✓ Function name `f` was changed to `add_numbers` to clearly describe its purpose.
- ✓ Variable names `a` and `b` were changed to `first_number` and `second_number` for clarity.
- ✓ Added a docstring to explain what the function does.
- ✓ Improved readability and maintainability without changing the program logic.

Task: 2

Problem Statement:

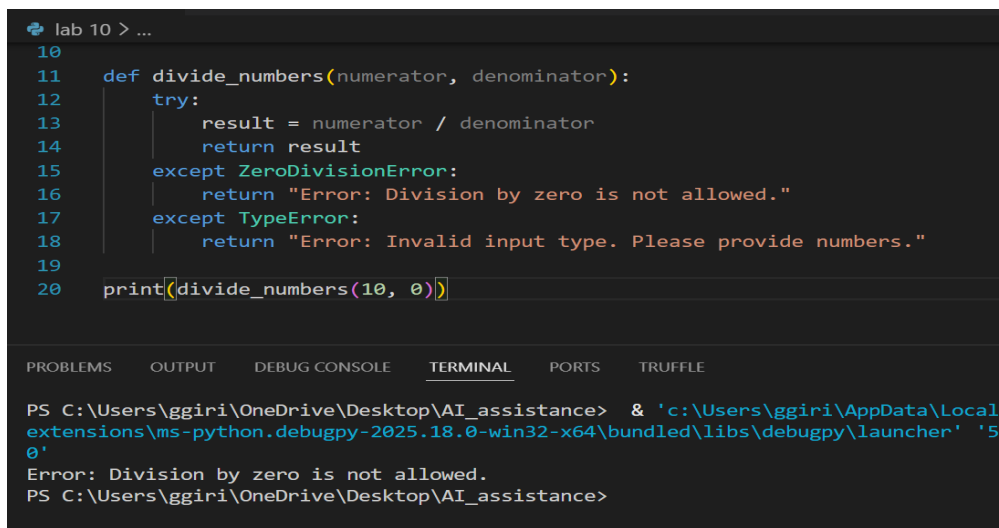
Given a Python function that performs division without error handling, improve the code by adding exception handling to prevent runtime errors such as division by zero.

Original Code:

```
def divide(a, b):  
  
    return a / b  
  
print(divide(10, 0))
```

Improved Code:

```
def divide_numbers(numerator, denominator):  
  
    try:  
  
        result = numerator / denominator  
  
        return result  
  
    except ZeroDivisionError:  
  
        return "Error: Division by zero is not allowed."  
  
    except TypeError:  
  
        return "Error: Invalid input type. Please provide numbers."  
  
print(divide_numbers(10, 0))
```



The screenshot shows a code editor with a dark theme. The top part displays the improved Python code for the `divide_numbers` function, which includes try-except blocks for `ZeroDivisionError` and `TypeError`. The bottom part shows the terminal output, where the command `python lab 10 > ...` is executed, and the output is `Error: Division by zero is not allowed.`

```
lab 10 > ...  
10  
11 def divide_numbers(numerator, denominator):  
12     try:  
13         result = numerator / denominator  
14         return result  
15     except ZeroDivisionError:  
16         return "Error: Division by zero is not allowed."  
17     except TypeError:  
18         return "Error: Invalid input type. Please provide numbers."  
19  
20 print(divide_numbers(10, 0))  
  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS TRUFFLE  
PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance> & 'c:\Users\ggiri\AppData\Local  
extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '5  
0'  
Error: Division by zero is not allowed.  
PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance>
```

Explanation of Changes:

- Renamed function `divide` to `divide_numbers` for clarity.
- Renamed variables `a`, `b` to `numerator`, `denominator`.
- Added try-except block to handle exceptions.

- Handled ZeroDivisionError to avoid program crash.
- Handled TypeError for invalid inputs.
- Provided clear and user-friendly error messages.

Task: 3

Objective:

To refactor code using AI for better readability and quality.

To follow PEP 8 coding standards.

To add proper variable names, functions, comments, and error handling.

Problem Statement:

Given a Python program that calculates total, average, and grade of a student with poor readability and no error handling, improve the code by restructuring it, adding meaningful names, and including validation and documentation.

Original code:

```
marks=[78,85,90,66,88]
```

```
t=0
```

```
for i in marks:
```

```
    t=t+i
```

```
a=t/len(marks)
```

```
if a>=90:
```

```
    print("A")
```

```
elif a>=75:
```

```
    print("B")
```

```
elif a>=60:
```

```
    print("C")
```

```
else:
```

```
    print("F")
```

Improved code:

```
lab 10 > ...
21
22 #task3
23
24 def calculate_total(marks_list):
25     """Calculate the total marks."""
26     return sum(marks_list)
27
28 def calculate_average(marks_list):
29     """Calculate the average marks."""
30     if len(marks_list) == 0:
31         raise ValueError("Marks list cannot be empty.")
32     return calculate_total(marks_list) / len(marks_list)
33
34 def calculate_grade(average_marks):
35     """Determine grade based on average marks."""
36     if average_marks >= 90:
37         return "A"
38     elif average_marks >= 75:
39         return "B"
40     elif average_marks >= 60:
41         return "C"
42     else:
43         return "F"
44
45 def validate_marks(marks_list):
46     """Validate that marks are numbers between 0 and 100."""
47     for mark in marks_list:
48         if not isinstance(mark, (int, float)):
49             raise TypeError("All marks must be numbers.")
50         if mark < 0 or mark > 100:
51             raise ValueError("Marks should be between 0 and 100.")
52
53 # Main Program
54 try:
55     student_marks = [78, 85, 90, 66, 88]
56     validate_marks(student_marks)
57     total_marks = calculate_total(student_marks)
58     average_marks = calculate_average(student_marks)
59     grade = calculate_grade(average_marks)
60
61     print("Total Marks:", total_marks)
62     print("Average Marks:", average_marks)
63     print("Grade:", grade)
64 except Exception as error:
65     print(f"Error: {error}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS TRUFFLE

```
Total Marks: 407
Average Marks: 81.4
Grade: B
PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance>
```

Explanation of Changes

- Renamed variables (t, a, i) to meaningful names (total_marks, average_marks, mark).
- Divided code into separate functions for modularity.
- Added **docstrings** for each function.
- Added **input validation** for marks.
- Included **exception handling** to manage errors.
- Used built-in functions like sum() for efficiency.
- Followed **PEP 8 naming conventions and formatting**.

Task: 4

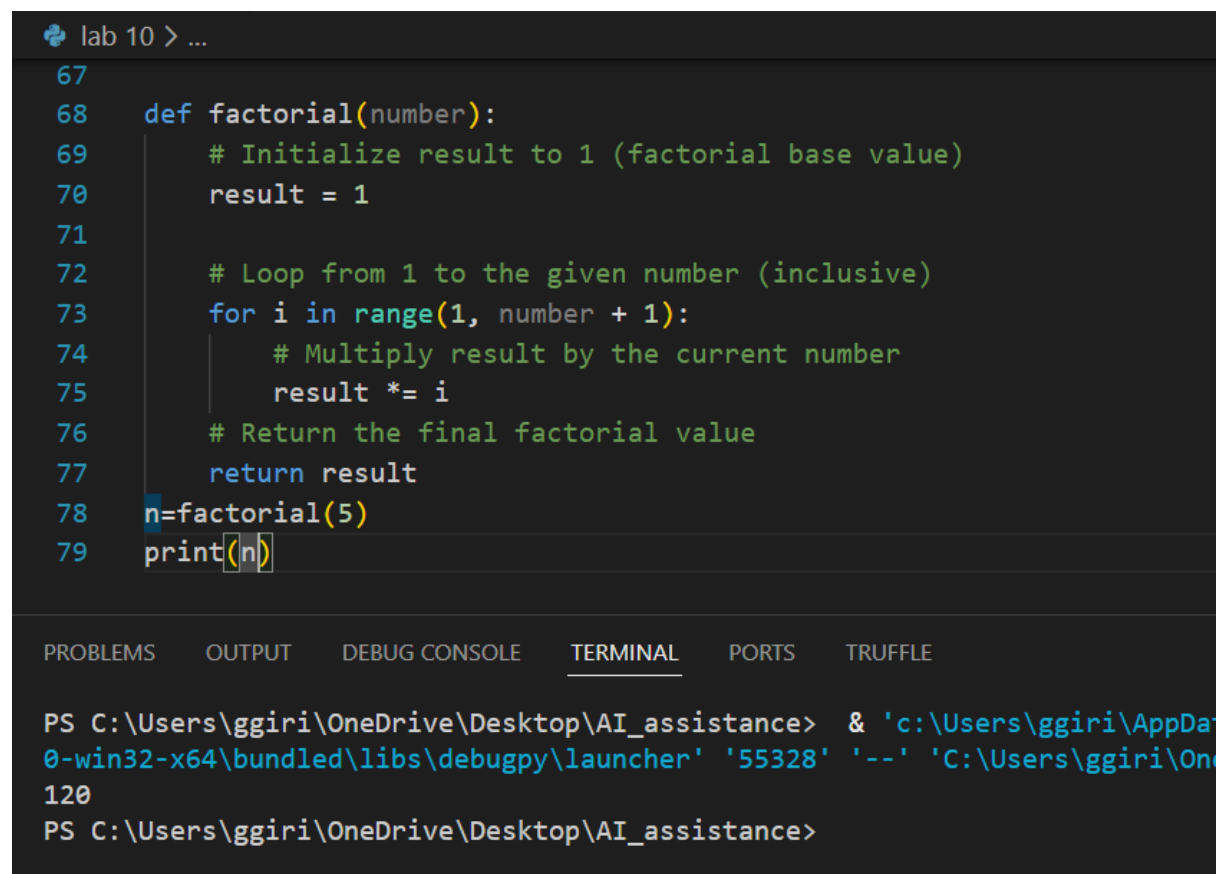
Problem Statement

Given a Python function without documentation or comments, improve the code by adding proper docstrings and inline comments to explain its functionality.

Original code:

```
def factorial(n):  
  
    result = 1  
  
    for i in range(1, n + 1):  
  
        result *= i  
  
    return result
```

Improved code:



```
lab 10 > ...  
67  
68 def factorial(number):  
69     # Initialize result to 1 (factorial base value)  
70     result = 1  
71  
72     # Loop from 1 to the given number (inclusive)  
73     for i in range(1, number + 1):  
74         # Multiply result by the current number  
75         result *= i  
76     # Return the final factorial value  
77     return result  
78 n=factorial(5)  
79 print(n)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS TRUFFLE

```
PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance> & 'c:\Users\ggiri\AppData  
0-win32-x64\bundled\libs\debugpy\launcher' '55328' '--' 'C:\Users\ggiri\One  
120  
PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance>
```

Task: 5

Problem Statement

The given program checks password strength only based on length, which is not secure. The task is to enhance validation by adding multiple conditions such as uppercase, lowercase, digit, and special character checks, and to improve code quality.

Original Code:

```
pwd = input("Enter password: ")
```

```
if len(pwd) >= 8:
```

```
    print("Strong")
```

```
else:
```

```
    print("Weak")
```

Improved Code:

```
lab 10 > is_strong_password

81  #task5
82
83  def is_strong_password(password):
84      # Check minimum length
85      if len(password) < 8:
86          return False
87
88      # Initialize flags
89      has_upper = False
90      has_lower = False
91      has_digit = False
92      has_special = False
93
94      # Define special characters
95      special_characters = "!@#$%^&*()-_+=[]{}|;: '\",.<>?/"
96
97      # Check each character in password
98      for char in password:
99          if char.isupper():
100             has_upper = True
101          elif char.islower():
102             has_lower = True
103          elif char.isdigit():
104             has_digit = True
105          elif char in special_characters:
106             has_special = True
107
108      # Return True only if all conditions are satisfied
109      return has_upper and has_lower and has_digit and has_special
110
111
112  # Main Program
113  user_password = input("Enter password: ")
114
115  if is_strong_password(user_password):
116      print("Strong Password")
117  else:
118      print("Weak Password")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  TRUFFLE

PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance> & 'c:\Users\ggiri\AppData\Local\F
0-win32-x64\bundled\libs\debugpy\launcher' '54240' '--' 'C:\Users\ggiri\OneDrive\Des
Enter password: @Giri2022
Strong Password
PS C:\Users\ggiri\OneDrive\Desktop\AI_assistance> |
```

Comparison: Original vs Improved:

Aspect	Original Code	Improved Code
Readability	Low	High
Structure	No functions	Modular functions
Maintainability	Hard to modify	Easy to update
Security	Only length check	Multiple security rules
Error Handling	None	Logical validation

Justification of Improvements:

- Minimum Length ensures password is not too short.
- Uppercase & Lowercase Letters increase complexity.
- Digits add variability.
- Special Characters make passwords harder to guess.
- Function-based design improves reusability.
- Clear naming and comments improve understanding.
- Overall, the enhanced version is more secure and follows software engineering best practices.

Conclusion:

Enhancing password validation significantly improves security and robustness. AI-assisted refactoring helps in writing clean, modular, and maintainable code while following standard coding practices.